Research Note

1. The learner problem

LLD practice is easy to start but hard to judge. A learner can sketch classes for a Parking Lot in 40 minutes and still not know if the responsibilities, abstractions and trade-offs hold up.

Three things make it hard:

- **Many valid answers.** A Strategy for pricing and a simple enum can both be right. Comparing against one reference solution marks a different-but-good design as wrong.
- **Good feedback is scarce.** The only reliable signal is a senior engineer or mock interviewer. That is costly and not repeatable every day.
- **No memory across attempts.** Learners rarely notice they repeat the same mistake (god classes, missing state handling, tight coupling) across problems.

What a learner actually needs: a clear problem, a structured way to show their design, feedback that points at specific parts of what they wrote, and a history that shows which weaknesses keep coming back.

2. Existing approaches

I looked at five common ways learners practise LLD today. Most either give no feedback on the learner's own design, or grade it against one reference answer.

Tool	How you practise	Submission	Feedback	Main limitation
Educative: Grokking the LLD Interview [3]	Read 16 case studies (Parking Lot, Chess, Library)	None	None on your design; you compare with the solution	Passive reading, no practice loop
awesome-low-level-design (GitHub, ~27k stars) [4]	Study reference implementations	None	Self-comparison	One reference reads as "the" answer
AlgoMaster LLD interviews [1]	AI-led mock on 45 problems: Requirements → Entities → Class design → Code	Chat + code editor	AI score /10 per stage, using a reference solution; history saved	Score is per stage, not per design quality; anchored to a reference
Codemia OOD [2]	Draw UML class diagrams	Diagram	"AI design review", editorial, progress dashboard	No published rubric; mostly HLD-focused
ChatGPT / Claude directly	Paste a design and ask "is this good?"	Free text	Unstructured opinion	Different answer every run, no history, often too agreeable

3. Key gaps

1. **Reference-anchored grading.** A valid design that differs from the model answer loses marks. The learner learns to copy, not to reason.
2. **Feedback without evidence.** "Improve cohesion" doesn't help unless it names the class and the line that caused it.
3. **One number, no direction.** A score of 6/10 doesn't tell the learner which skill to work on next.
4. **No pattern across attempts.** No tool I saw tells a learner "you missed state handling in 3 of your last 4 designs".

5. **Extensibility is never tested.** The real LLD skill is surviving a requirement change, yet practice ends at the first submission.

4. Product direction

Grade the design against the requirements, not against a model answer, and make the learner face a requirement change. Each gap above maps to one decision:

Gap	MVP decision
Reference-anchored grading	Each problem has a checklist of required behaviours and a fixed 6-dimension rubric. Any design that meets them can score well.
Feedback without evidence	Every finding must quote the part of the submission it refers to. Findings without a quote are dropped.
One number, no direction	A score per rubric dimension, plus the single most important thing to fix next.
No pattern across attempts	History shows the learner's weakest dimension across all attempts.
Extensibility never tested	After the first attempt, the retry adds a change request (e.g. "add EV charging spots"). Feedback checks how much of the design had to change.

Submission format: structured text plus key interfaces as code. The learner fills five short sections: requirements and assumptions, classes and their responsibilities, key interfaces (code), main flow, and trade-offs. This is the smallest format that shows both reasoning and abstraction. Diagrams add build effort but little extra evidence for an MVP.

Out of scope: diagram editors, running learner code, timers, peer review, courses.

Sources

1. AlgoMaster, LLD interview practice. https://algomaster.io/interview/low-level-design (accessed 13 Oct 2026)
2. Codemia, system design and OOD practice. https://www.codemia.io (accessed 13 Oct 2026)
3. Javarevisited, review of Educative's "Grokking the Low Level Design Interview". https://medium.com/javarevisited/review-is-grokking-the-low-level-design-interview-using-ood-principles-on-educative-worth-it-c49e3c65d7de (accessed Oct 2026)
4. awesome-low-level-design on HelloGitHub. https://hellogithub.com/en/repository/ashishps1/awesome-low-level-design (accessed 13 Oct 2026)